

Traitement du TP1

Rédigé par :
Karel SODJINOUTI

2026-04-15

Table des matières

1	Exercice 1 — Typage implicite, coercition et ambiguïtés	3
1.1	Déterminer, sans exécuter le code, le type et le mode de chaque objet.	3
1.2	Analyse du résultat des expressions :	3
1.3	Identifier tous les cas dans lesquels R effectue une coercition implicite.	4
1.4	Préciser, pour chaque cas, le type cible et la logique de conversion mobilisée.	4
1.5	Discuter rigoureusement les conséquences de l'instruction suivante :	5
2	Exercice 2 — Structures hétérogènes et propagation des types	5
2.1	Détermination du type de chaque colonne de D. .	6
2.2	Expliquer précisément pourquoi R accepte cette structure malgré l'hétérogénéité initiale	6
2.3	Comparer le comportement de l'objet D avec celui obtenu après l'instruction suivante :	6
2.4	Détermination du type final de la matrice obtenue et justification formelle de la réponse.	7

2.5	Montrer en quoi cette transformation peut entraîner une perte d'information statistique dans un contexte d'analyse de données.	7
2.6	Proposer une stratégie permettant de préserver les types tout en rendant possibles certaines opérations de nature matricielle.	7
3	Exercice 3 — Facteurs, encodage et biais d'interprétation	8
3.1	Transformer niveau en facteur ordonné avec une hiérarchie pertinente.	8
3.2	Déterminer les valeurs numériques internes associées aux modalités.	8
3.3	Expliquer pourquoi l'instruction suivante peut être dangereuse dans une analyse :	8
3.4	Proposer une méthode correcte pour obtenir une représentation numérique cohérente.	9
3.5	Discussion de la validité statistique	9
3.6	Identifier un biais potentiel si les niveaux du facteur sont mal ordonnés.	10
4	Exercice 4 Construction structurée d'un data frame	10
4.1	Créer une variable moyenne_annuelle en tenant compte des cas incomplets.	11
4.2	Variable Issue :	11
4.3	Pertinence de transformer la variable filiere en facteur ordonné	12
4.4	Implémenter un contrôle automatique permettant de détecter : les incohérences de type; les valeurs aberrantes; les valeurs manquantes critiques.	12
4.5	Discuter les implications de ces choix de type pour une régression linéaire, une analyse factorielle.	17

1 Exercice 1 — Typage implicite, coercion et ambiguïtés

1.1 Déterminer, sans exécuter le code, le type et le mode de chaque objet.

x est de type ("integer") et de mode ("numeric") y est de type ("logical") et de mode ("logical") z est de type ("character") et de mode ("character") w est de type ("list") et de mode ("list")

1.2 Analyse du résultat des expressions :

```
x <- c(1, 2, 3)
y <- c(TRUE, FALSE, TRUE)
z <- c("4", "5", "6")
w <- list(1, "2", TRUE)
y+as.integer(w)
```

```
[1] 2 2 2
```

— x+y :

```
“ r x+y “
```

```
“ [1] 2 2 4 “
```

x est de type "integer" et y de classe "logical". Mais puisque le type integer est le supérieur hiérarchique de logical, le vecteur x devient un integer et la somme se fait un à un entre integer

— x+z : **Résultat** Erreur dans x + z : argument non numérique pour un opérateur binaire

Dans ce processus, x, est obligé de devenir de type caractère. Mais puisque on ne peut additionner (au sens mathématique) des chaînes de caractère (elles sont stockés comme telle par R), R dit qu'il ne peut le faire.

— : as.numeric(z) + x

```
“ r as.numeric(z) + x “
```

“ [1] 5 7 9 “ Ici, z devient un vecteur numérique. Il a au moins le type double (auquel cas x devient double) et donc, l'addition peut se faire entre z et x .

- $\text{unlist}(w) + x$ **Résultat** : Erreur dans $\text{unlist}(w) + x$: argument non numérique pour un opérateur binaire

Quand on enlève w au format liste, il devient un atomic vector contenant : 1, "2" et TRUE. Par supériorité hiérarchique dans la classe des éléments qui composent un vecteur, le vecteur w devient de type "character". Dans l'addition, x devient aussi caractère pour que R puisse additionner des objets de même type. Et comme vu en deuxième point, on ne peut additionner des chaînes de caractère.

1.3 Identifier tous les cas dans lesquels R effectue une coercition implicite.

- $x + y$
- $x + z$
- $x + \text{unlist}(z)$
- $y + \text{as.integer}(z)$
- $y + \text{as.integer}(w)$

1.4 Préciser, pour chaque cas, le type cible et la logique de conversion mobilisée.

- $x + y$: y devient un vecteur de type "integer"
- $x + z$: x devient un vecteur de type "character"
- $x + \text{unlist}(z)$: $\text{unlist}(z)$ pousse la liste à devenir un vecteur avec comme type celui du plus grand dans la hiérarchie à savoir "character". L'addition va obliger x à être de type "character".
- $y + \text{as.integer}(z)$: $\text{as.integer}(z)$ transforme z en un vecteur d'entiers : "integer". y qui était "logical" devient donc "integer" à cause de l'addition qui se fait sur des vecteurs de même type.

- `y+as.integer(w)` : `as.integer` converti `w` en entier. A cause de l'addition, `y` devient de type `integer`. Et l'addition se fait.

1.5 Discuter rigoureusement les conséquences de l'instruction suivante :

```
as.numeric(c("10", "A", "20"))
```

```
[1] 10 NA 20
```

Cette instruction oblige qu'on prenne en compte des NA dans les calculs qui seront fait impliquant ce vecteur. Si cela n'est pas pris en compte, alors les résultats risquent d'être biaisés.

##Proposer une méthode robuste permettant de détecter automatiquement les conversions problématiques dans un vecteur.

```
x <- c("10", "A", "20", "B")
y <- as.numeric(x)
probleme <- x[is.na(y)]
indice <- which(is.na(y))
cat(probleme, " qui est/sont dans x à la/les position (s) ",
```

```
A B qui est/sont dans x à la/les position (s) 2 4 n'est (n
```

2 Exercice 2 — Structures hétérogènes et propagation des types

On considère la construction suivante :

```
A <- c(1, 2, 3)
B <- c("a", "b", "c")
C <- c(TRUE, FALSE, TRUE)
D <- data.frame(A, B, C)

typeof(D[,A]) # "list"
```

```
[1] "list"
```

```
typeof(D[,C]) # "list"
```

```
[1] "list"
```

```
class(D)
```

```
[1] "data.frame"
```

2.1 Détermination du type de chaque colonne de D.

Toutes les colonnes sont de type “list”.

2.2 Expliquer précisément pourquoi R accepte cette structure malgré l’hétérogénéité initiale

Un dataframe est stocké sous forme de liste. Or une liste est capable de contenir plusieurs types de données tout en préservant ce type. D’où le fait que R accepte cette structure.

2.3 Comparer le comportement de l’objet D avec celui obtenu après l’instruction suivante :

```
as.matrix(D)
```

```
      A    B    C  
[1,] "1"  "a" "TRUE"  
[2,] "2"  "b" "FALSE"  
[3,] "3"  "c" "TRUE"
```

```
D
```

```
      A B      C  
1 1 a  TRUE  
2 2 b FALSE  
3 3 c  TRUE
```

```
m <- as.matrix(D)
```

On remarque qu’un objet de classe data.frame préserve le type des éléments de ses colonnes en les mettant dans une liste ce qui n’est pas le cas avec la matrice.

```
mean(D[,1])
```

```
[1] 2
```

2.4 Détermination du type final de la matrice obtenue et justification formelle de la réponse.

```
typeof(m)
```

```
[1] "character"
```

La matrice obtenue est de type “character”. En effet une matrice effectue une coercion suivant le type sur tout les éléments qui la compose.

2.5 Montrer en quoi cette transformation peut entraîner une perte d’information statistique dans un contexte d’analyse de données.

D’abord, le fait que la transformation coerce tous les éléments de la matrice dans le type le plus élevé dans la hiérarchie fait perdre le type de données. Dans un dataframe contenant plusieurs, type de données, vous ne pouvez que manipuler un seul type. \ Quand vous calculer une statistique sur des colonnes quantitatives (comme colMeans()), vous aurez des NA au lieu de vraies valeurs.

2.6 Proposer une stratégie permettant de préserver les types tout en rendant possibles certaines opérations de nature matricielle.

On peut d’abord convertir la matrice au format dataframe. Cela permet de préserver la structure des données. A la suite de cela, on extrait les données numériques qui dans un dataframe nommé df_num par exemple. On peut maintenant convertir df_num au format matricielle et effectuer les opérations matricielles voulues.

3 Exercice 3 — Facteurs, encodage et biais d'interprétation

On considère le vecteur suivant :

```
niveau <- c("faible", "moyen", "élevé", "moyen", "faible")
```

3.1 Transformer niveau en facteur ordonné avec une hiérarchie pertinente.

```
niveau <- factor(niveau, levels = c("faible", "moyen", "élevé"))
```

3.2 Déterminer les valeurs numériques internes associées aux modalités.

```
as.integer(niveau)
```

```
[1] 1 2 3 2 1
```

3.3 Expliquer pourquoi l'instruction suivante peut être dangereuse dans une analyse :

```
as.numeric(niveau)
```

```
[1] 1 2 3 2 1
```

Elle peut être dangereuse dans le cas où nous avons des nombres dans le facteur et que nous voulions voir le visage de ces nombres. Quand on met cette commande, elle nous donne les codes utilisés par R pour encoder ces nombres au lieu de la valeur réelle de ces nombres même. Par exemple :

```
x <- factor(c(3678, 2, 4))  
as.numeric(x)
```

```
[1] 3 1 2
```

3.4 Proposer une méthode correcte pour obtenir une représentation numérique cohérente.

On converti d'abord le factor en une chaîne de caractère. Et après cela, on le converti en numeric. Par exemple

```
x <- factor(c(3678,2,4))
as.numeric(as.character(x))
```

```
[1] 3678      2      4
```

Différent de :

```
x <- factor(c(3678,2,4))
as.numeric(x)
```

```
[1] 3 1 2
```

3.5 Disucussion de la validité statistique

On définit le vecteur suivant :

```
revenu <- c(100, 200, 300, 250, 150)
```

Discuter la validité statistique de l'expression :

```
mean(revenu[niveau == "élevé"])
```

```
[1] 300
```

Cette opération est valide si les deux vecteurs : revenu et niveau ont la même longueur. Par ailleurs, si "élevé" n'était pas dans les levels, même s'il était dans les éléments qui ont construit le factor (les labels), le calcul précédent retournerait un NA comme ci dessous :

```
niv <- c("faible", "moyen", "élevé", "moyen", "faible")
niv <- factor(niv, levels = c("faible", "moyen"))
mean(revenu[niv == "élevé"])
```

```
[1] NA
```

3.6 Identifier un biais potentiel si les niveaux du facteur sont mal ordonnés.

Biais dans le rangement : on peut avoir des opérateurs de comparaison faussés

```
# Facteur ordonné mal configuré
niveau_etude <- ordered(c("Lycée", "Master", "Licence"),
                        levels = c("Lycée", "Master", "Licence"))
#                               Master avant Li

niveau_etude[1] < niveau_etude[2]  # TRUE   Lycée < Master

[1] TRUE

niveau_etude[2] < niveau_etude[3]  # TRUE   Master < Licence

[1] TRUE

# Correction
niveau_etude <- ordered(c("Lycée", "Master", "Licence"),
                        levels = c("Lycée", "Licence", "Master"))
niveau_etude[2] < niveau_etude[3]  # FALSE  Licence < Master

[1] FALSE
```

4 Exercice 4 Construction structurée d'un data frame

On dispose des informations suivantes sur une cohorte d'étudiants :

- identifiant unique
- sexe
- filière (ISEP, ISE, AS)
- moyennes semestrielles S1 et S2
- statut de boursier
- région d'origine, avec certaines valeurs manquantes

Construire un data.frame cohérent

```
base <- data.frame(
  identifiant=c("S01","S02","S03","S04","S05"),
  sexe=rep_len(c("M","F"),5),
  filière=rep_len(c("ISEP","ISE","AS"),5),
  moyenne_S1=c(14,11,12,NA,NA),
  moyenne_S2=c(16,20,NA,15,13),
  statut_boursier=rep_len(c(TRUE,FALSE),5),
  region_d_origine=c("Kolda",NA,"Kaffrine",NA,"Tamba")
)
```

4.1 Créer une variable moyenne_annuelle en tenant compte des cas incomplets.

```
base$moyenne_annuelle <- with(base, (base$moyenne_S1+base$moyenne_S2)/2)
base
```

	identifiant	sexe	filière	moyenne_S1	moyenne_S2	statut_boursier
1	S01	M	ISEP	14	16	TRUE
2	S02	F	ISE	11	20	FALSE
3	S03	M	AS	12	NA	FALSE
4	S04	F	ISEP	NA	15	FALSE
5	S05	M	ISE	NA	13	FALSE

	region_d_origine	moyenne_annuelle
1	Kolda	15.0
2	<NA>	15.5
3	Kaffrine	NA
4	<NA>	NA
5	Tamba	NA

4.2 Variable Issue :

Créer une variable qualitative prenant la modalité

- "Admis" si la moyenne annuelle est supérieure ou égale à 12;
- "Ajourné" sinon.

```
base$issue <- ifelse(base$moyenne_annuelle >= 12, "Admis", "Ajo
```

4.3 Pertinence de transformer la variable filiere en facteur ordonné

Oui il est pertinent de le faire. Cela permet de d'éviter le biais dans le rangement des modalités et de ne pas fausser les opérateurs de comparaison, comme vu précédemment. ## Vérifier la structure complète de l'objet et commenter les types, la cohérence et les éventuelles anomalies

```
names(base)
```

```
[1] "identifiant"      "sexe"             "filière"
[5] "moyenne_S2"       "statut_boursier"  "region_d_origine"
[9] "issue"
```

```
for (i in seq_along(names(base))) print(typeof(base[,i]))
```

```
[1] "character"
[1] "character"
[1] "character"
[1] "double"
[1] "double"
[1] "logical"
[1] "character"
[1] "double"
[1] "character"
```

Le type est cohérent avec les variables que nous avons créées. il n'y a pas d'anomalie.

4.4 Implémenter un contrôle automatique permettant de détecter : les incohérences de type; les valeurs aberrantes; les valeurs manquantes critiques.

```
controler_dataframe <- function(df,
                                seuil_outlier = 3,
```

```

seuil_manquant = 0.1) {

rapport <- list()

#
# 1. INCOHÉRENCES DE TYPE
#
rapport$types <- lapply(names(df), function(col) {
  x <- df[[col]]

  incoherences <- c()

  # Colonne numérique contenant des chaînes non convertibles
  if (is.character(x)) {
    tentative <- suppressWarnings(as.numeric(x))
    n_coercibles <- sum(!is.na(tentative))
    if (n_coercibles > length(x) * 0.8) {
      incoherences <- c(incoherences,
        paste0(" '", col, "' est character mais ",
          n_coercibles, " valeurs sont numériques"))
    }
  }

  # Facteur avec niveaux inattendus (non déclarés)
  if (is.factor(x)) {
    niveaux_obs <- levels(droplevels(x))
    niveaux_dec <- levels(x)
    fantomes <- setdiff(niveaux_dec, niveaux_obs)
    if (length(fantomes) > 0) {
      incoherences <- c(incoherences,
        paste0(" '", col, "' a des niveaux fantômes : ",
          paste(fantomes, collapse = ", ")))
    }
  }

  # Colonne logique stockée en integer (0/1)
  if (is.integer(x) && all(x %in% c(0L, 1L, NA))) {
    incoherences <- c(incoherences,

```

```

        paste0(" '", col,
               "' pourrait être logical (contient seulement 0)
    }

    if (length(incoherences) == 0) NULL else incoherences
})
names(rapport$types) <- names(df)
rapport$types <- Filter(Negate(is.null), rapport$types)

#
# 2. VALEURS ABERRANTES (méthode Z-score)
#
rapport$outliers <- lapply(names(df), function(col) {
  x <- df[[col]]
  if (!is.numeric(x)) return(NULL)

  x_clean <- x[!is.na(x)]
  if (length(x_clean) < 3) return(NULL)

  z_scores <- abs((x_clean - mean(x_clean)) / sd(x_clean))
  idx_outliers <- which(z_scores > seuil_outlier)

  if (length(idx_outliers) == 0) return(NULL)

  data.frame(
    colonne = col,
    indice = idx_outliers,
    valeur = x_clean[idx_outliers],
    z_score = round(z_scores[idx_outliers], 2)
  )
})
names(rapport$outliers) <- names(df)
rapport$outliers <- Filter(Negate(is.null), rapport$outliers)

#
# 3. VALEURS MANQUANTES CRITIQUES
#

```

```

rapport$manquants <- lapply(names(df), function(col) {
  x <- df[[col]]
  n <- length(x)
  na <- sum(is.na(x))
  taux <- na / n

  if (na == 0) return(NULL)

  gravite <- dplyr::case_when(
    taux >= 0.5 ~ "CRITIQUE ( 50%)",
    taux >= 0.2 ~ "ÉLEVÉ ( 20%)",
    taux >= seuil_manquant ~ "MODÉRÉ ( 10%)",
    TRUE ~ "FAIBLE (< 10%)"
  )

  data.frame(
    colonne = col,
    n_NA = na,
    taux_NA = paste0(round(taux * 100, 1), "%"),
    gravite = gravite
  )
})
names(rapport$manquants) <- names(df)
rapport$manquants <- Filter(Negate(is.null), rapport$manquants)

#
# AFFICHAGE DU RAPPORT
#
cat("\n")
cat(" RAPPORT DE CONTRÔLE QUALITÉ\n")
cat("\n\n")

cat(" 1. INCOHÉRENCES DE TYPE \n")
if (length(rapport$types) == 0) {
  cat(" Aucune incohérence détectée\n")
} else {
  lapply(rapport$types, function(x) cat(" ", x, "\n"))
}

```

```

}

cat("\n 2. VALEURS ABERRANTES (|Z| >",
    seuil_outlier, ") \n")
if (length(rapport$outliers) == 0) {
  cat("    Aucune valeur aberrante détectée\n")
} else {
  print(do.call(rbind, rapport$outliers))
}

cat("\n 3. VALEURS MANQUANTES CRITIQUES \n")
if (length(rapport$manquants) == 0) {
  cat("    Aucune valeur manquante\n")
} else {
  print(do.call(rbind, rapport$manquants))
}

invisible(rapport)
}

df_test <- data.frame(
  age      = c(25, 30, NA, 999, 28),          # outlier + NA
  salaire = c("3000", "4500", "abc", NA, "3800"), # incohérence
  niveau  = factor(c("Junior", "Senior", NA,
                     "Junior", "Expert"),
                  levels = c("Junior", "Senior",
                             "Expert", "Manager")), # niveau
  actif   = c(1L, 0L, 1L, 1L, 0L)           # logical déguisé
)

rapport <- controler_dataframe(df_test, seuil_outlier = 2.5)

```

RAPPORT DE CONTRÔLE QUALITÉ

1. INCOHÉRENCES DE TYPE

'niveau' a des niveaux fantômes : Manager
 'actif' pourrait être logique (contient seulement 0/1)

2. VALEURS ABERRANTES ($|Z| > 2.5$)

Aucune valeur aberrante détectée

3. VALEURS MANQUANTES CRITIQUES

	colonne	n_NA	taux_NA		gravite
age	age	1	20%	ÉLEVÉ	(20%)
salaire	salaire	1	20%	ÉLEVÉ	(20%)
niveau	niveau	1	20%	ÉLEVÉ	(20%)

4.5 Discuter les implications de ces choix de typage pour une régression linéaire, une analyse factorielle.

implication pour la régression linéaire : - numeric bien typé, les coefficients sont interprétables directement - character non converti, R refuse ou coerce silencieusement biais - factor non ordonné crée des variables dummy automatiquement (k-1 indicatrices) - factor mal ordonné donne une mauvaise baseline coefficients inversés

```
df2 <- data.frame(
  niveau = factor(c("Junior", "Senior", "Manager", "Junior",
                    levels = c("Junior", "Senior", "Manager"))),
  salaire = c(30000, 50000, 75000, 32000, 52000)
)
# Un factor crée automatiquement des indicatrices
df2$niveau <- factor(df2$niveau,
  levels = c("Junior", "Senior", "Manager"))
lm(salaire ~ niveau, data = df2)
```

Call:

```
lm(formula = salaire ~ niveau, data = df2)
```

Coefficients:

```
(Intercept)  niveauSenior  niveauManager
```

31000

20000

44000

```
# R génère : niveauSenior, niveauManager
# Référence implicite = "Junior" (1er niveau)
# Si l'ordre est mauvais, l'interprétation est fausse
```

Pour une ACP numeric homogène donne une matrice de corrélation valide ACP fiable Types mixtes non traités entraîne que cor() échoue ACP impossible factor inclus brut génère une Erreur ou résultat incohérent

```
df <- df2
# Bonne pratique avant une ACP
df_num <- df[, sapply(df, is.numeric)] # 1. Isoler le numérique
df_num <- na.omit(df_num) # 2. Supprimer les NA
df_scale <- scale(df_num) # 3. Standardiser

# ACP propre
acp <- prcomp(df_scale, center = TRUE, scale. = TRUE)
```